

Отчет по выполнению 4 лабораторной работы.

Филонов Иван 8 группа

1. Практическое задание №4 - Helm, CRD & RBAC

1.1. Введение

Целью данной работы является автоматизация развертывания облачной инфраструктуры с использованием Helm-чартов, операторов и принципов декларативного управления Kubernetes. В ходе выполнения решаются задачи по внедрению CloudNativePG для управления базой данных, настройке Cert-Manager для обеспечения безопасности через TLS-сертификаты. Итогом работы является создание единого, параметризованного пакета приложения, готового к развертыванию в кластере одной командой.

1.2. Ход работы

Для начала нужно установить необходимые операторы.

```
C:\Users\iafil>winget install Helm.Helm
Найдено Helm [Helm.Helm] Версия 4.2.0
Лицензия на это приложение предоставлена вам владельцем.
Корпорация Майкрософт не несет ответственность за сторонние пакеты и не предоставляет для них никакие лицензии.
Скачивание https://get.helm.sh/helm-v4.2.0-windows-amd64.zip
19.3 MB / 19.3 MB
Хэш установщика успешно проверен
Извлечение архива...
Архив извлечен
Запуск установки пакета...
Переменная среды пути изменена; перезапустите оболочку, чтобы использовать новое значение.
Добавлен псевдоним командной строки: "helm"
Успешно установлено
```

Установим CloudNativePG (оператор для PostgreSQL).

```
C:\Users\iafil>helm repo add cnpg https://cloudnative-pg.github.io/charts
"cnpg" has been added to your repositories
```

Установим Cert-Manager (управление TLS-сертификатами).

```
C:\Users\iafil>helm upgrade --install cnpg cnpg/cloudnative-pg --namespace cnpg-system --create-namespace
Release "cnpg" does not exist. Installing it now.
NAME: cnpg
LAST DEPLOYED: Fri May 15 22:38:23 2026
NAMESPACE: cnpg-system
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
NOTES:
CloudNativePG operator should be installed in namespace "cnpg-system".
You can now create a PostgreSQL cluster with 3 nodes as follows:

cat <<EOF | kubectl apply -f -
# Example of PostgreSQL cluster
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: cluster-example

spec:
  instances: 3
  storage:
    size: 1Gi
EOF

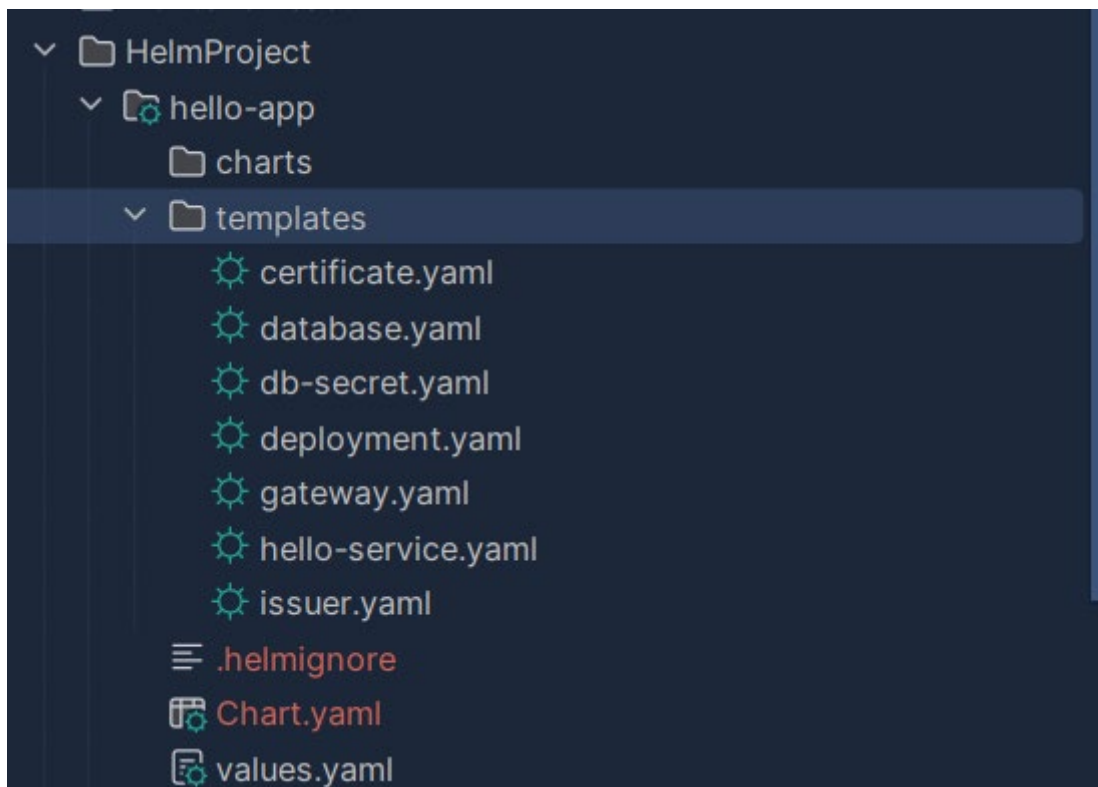
kubectl get -A cluster

C:\Users\iafil>
C:\Users\iafil>helm repo add jetstack https://charts.jetstack.io
"jetstack" has been added to your repositories
```

Дальше создадим новую папку HelmProject и инициализируем чарт.

```
PS C:\Users\iafil\OneDrive\Документы\course3\devops\devops-practice1-docker-compose>
PS C:\Users\iafil\OneDrive\Документы\course3\devops\devops-practice1-docker-compose> cd .\HelmProject\
PS C:\Users\iafil\OneDrive\Документы\course3\devops\devops-practice1-docker-compose\HelmProject> helm create hello-app
Creating hello-app
PS C:\Users\iafil\OneDrive\Документы\course3\devops\devops-practice1-docker-compose\HelmProject>
```

Далее в папку templates мы добавим необходимые манифесты(перепишем из предыдущих лабораторных)



Переделаем базу данных на CloudNativePG, используем ресурс Cluster.

```
database.yaml x
1  apiVersion: postgresql.cnpg.io/v1
2  kind: Cluster
3  metadata:
4    name: {{ .Values.db.name }}
5  spec:
6    instances: 1
7    storage:
8      size: 1Gi
9    bootstrap:
10     initdb:
11       database: app
12       owner: app
13       secret:
14         name: my-postgres-db-custom-password
```

Далее настроим HTTPS с Cert-Manager. Создадим самописный центр сертификации и сам сертификат.

```
issuer.yaml x
1  apiVersion: cert-manager.io/v1
2  kind: ClusterIssuer
3  metadata:
4    name: selfsigned-issuer
5  spec:
6    selfSigned: {}
```

```
certificate.yaml x
1  apiVersion: cert-manager.io/v1
2  kind: Certificate
3  metadata:
4    name: hello-app-cert
5  spec:
6    secretName: hello-app-tls
7    issuerRef:
8      name: selfsigned-issuer
9      kind: ClusterIssuer
10   dnsNames:
11     - "localhost"
```

Обновим Gateway, добавим TLS

```
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
spec:
  gatewayClassName: eg
  listeners:
    - name: https
      protocol: HTTPS
      port: 443
      tls:
        mode: Terminate
        certificateRefs:
          - name: hello-app-tls
      allowedRoutes:
        namespaces:
          from: Same
```

Шаблонизируем приложение, модифицируем Deployment

```
certificate.yaml gateway.yaml deployment.yaml x
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: hello-deployment
5  spec:
6    replicas: 1
7    selector:
8      matchLabels:
9        app: hello
10   template:
11     metadata:
12       labels:
13         app: hello
14     spec:
15       containers:
16         - name: hello-app
17           image: "{{ .Values.image.repository }}:v3"
18           ports:
19             - containerPort: 5001
20           env:
21             - name: NAME
22               value: "{{ Ivan | quote }}"
23             - name: DB_HOST
24               value: "{{ .Values.db.name }}-rw"
25             - name: DB_NAME
26               value: "app"
27             - name: DB_USER
28               value: "app"
29             - name: DB_PASSWORD
30               valueFrom:
31                 secretKeyRef:
32                   name: "{{ .Values.db.name }}-custom-password"
33                   key: password
```

Заполним Values, здесь будут все основные настройки.

```
values.yaml × certificate.yaml gateway.ya
1      replicaCount: 1
2      appName: "Ivan"
3
4      image:
5        repository: fokus00/hello-python
6        tag: "v3"
7
8      db:
9        name: my-postgres-db
10       customPassword: "secretpassword"
11
12     issuerRef:
13       name: selfsigned-issuer
14
```

Для использования кастомного пароля БД опишем манифест секрета и шаблонизируем его. Пароль указываем в Values.

```
values.yaml db-secret.yaml × certificate.yaml ga
1      apiVersion: v1
2      kind: Secret
3      metadata:
4        name: {{ .Values.db.name }}-custom-password
5      type: kubernetes.io/basic-auth
6      stringData:
7        username: "app"
8        password: {{ .Values.db.customPassword | quote }}
```

Осталось запустить и проверить работу. Для этого устанавливаем нужные операторы.

Команды для этого:

```

# CloudNativePG

```
helm upgrade --install cnpg cnpg/cloudnative-pg --namespace
cnpg-system --create-namespace
```

# Cert-Manager

```
helm upgrade --install cert-manager jetstack/cert-manager --
namespace cert-manager --create-namespace --set installCRDs=true
'''
```

После этого запускаем.

'''

```
helm upgrade --install my-hello-release ./hello-app
```

'''

```
PS C:\Users\iafil\OneDrive\Документы\course3\devops\devops-practice1-docker-compose\HelmProject> helm upgrade --install my-hello-release ./hello-app
Release "my-hello-release" has been upgraded. Happy Helming!
NAME: my-hello-release
LAST DEPLOYED: Mon Jun 1 20:50:33 2026
NAMESPACE: default
STATUS: deployed
REVISION: 5
DESCRIPTION: Upgrade complete
TEST SUITE: None
PS C:\Users\iafil\OneDrive\Документы\course3\devops\devops-practice1-docker-compose\HelmProject>
```

Для корректной работы запускаем туннелирование.

```
C:\Windows\System32>minikube tunnel
* Tunnel successfully started

* NOTE: Please do not close this terminal as this process must stay alive for the tunnel to be accessible ...

! Access to ports below 1024 may fail on Windows with OpenSSH clients older than v8.1. For more information, see: https://minikube.sigs.k8s.io/docs/handbook/accessing/#access-to-ports-1024-on-windows-requires-root-permission
* Starting tunnel for service envoy-default-my-gateway-1c7c06f0.
! Access to ports below 1024 may fail on Windows with OpenSSH clients older than v8.1. For more information, see: https://minikube.sigs.k8s.io/docs/handbook/accessing/#access-to-ports-1024-on-windows-requires-root-permission
* Starting tunnel for service hello-ingress.
```

Проверяем

```
C:\Users\iafil>kubectl get gateway
NAME CLASS ADDRESS PROGRAMMED AGE
my-gateway eg 127.0.0.1 True 16d

C:\Users\iafil>|
```

Открываем сервис

```
C:\Users\iafil>minikube service hello-service
```

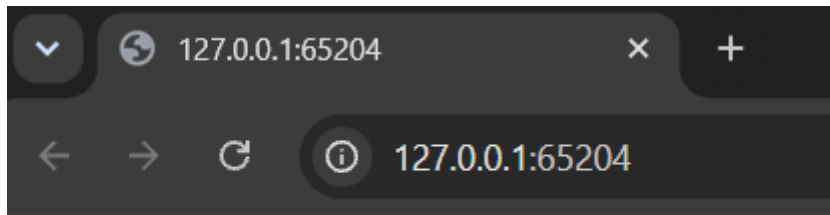
| NAMESPACE | NAME          | TARGET PORT | URL          |
|-----------|---------------|-------------|--------------|
| default   | hello-service |             | No node port |

```
* service default/hello-service has no node port
! Services [default/hello-service] have type "ClusterIP" not meant to be exposed, however for local development minikube
allows you to access this !
* Starting tunnel for service hello-service.
```

| NAMESPACE | NAME          | TARGET PORT | URL                    |
|-----------|---------------|-------------|------------------------|
| default   | hello-service |             | http://127.0.0.1:65204 |

```
* Opening service default/hello-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Открывается рабочая страница как в предыдущей лабораторной.



# Hello Ivan!

This page has been viewed 3 times.

## 2. Вывод

В ходе выполнения практической работы была успешно реализована автоматизированная система развертывания приложения с использованием экосистемы Helm и операторов Kubernetes. Внедрение CloudNativePG позволило перевести управление базой данных на уровень декларативных манифестов, обеспечив отказоустойчивость и автоматизацию жизненного цикла данных. Интеграция Cert-Manager и Gateway API обеспечила соответствие современным стандартам безопасности, позволив настроить защищенный HTTPS-доступ к сервису. Итогом работы стало создание гибкого, легко масштабируемого Helm-чарта, который унифицирует развертывание всех компонентов инфраструктуры в кластере, существенно снижая вероятность ошибок при конфигурировании и сокращая время ввода системы в эксплуатацию.